

Rendu individuel

Adame Nianghane
Observabilité & sécurité

Auteur	Adame Nianghane
Rôle	Observabilité & sécurité
Classe / Promo	M2 DO C — 2025-2026
Période	Janvier – Juin 2026
Projet	Plateforme de gestion d'un cabinet d'audioprothèse
Domaine	Supervision (Prometheus/Grafana/Loki), DevSecOps, durcissement

SUP DE VINCI — Expert en Systèmes d'Information — Mastère DevOps — Classe **M2 DO C** — Promotion 2025-2026. Document réalisé dans le cadre du projet d'étude de fin d'année.

1. Contexte et rôle dans le projet

J'ai pris en charge deux volets complémentaires qui déterminent la confiance que l'on peut accorder à un système en production : l'**observabilité**, c'est-à-dire la capacité à comprendre ce que fait la plateforme à tout instant, et la **sécurité**, tant celle de la chaîne logicielle que celle de l'exécution. Ces deux sujets partagent une même philosophie : anticiper les problèmes plutôt que les subir, en rendant le système transparent et en le durcissant à chaque étape.

Ce positionnement m'a conduit à intervenir sur l'ensemble du périmètre : j'ai instrumenté l'application développée par Zaafir, intégré mes analyses de sécurité dans la chaîne d'Elyess, et sécurisé les objets Kubernetes packagés par Anis. L'observabilité et la sécurité sont en effet des préoccupations transverses, qui ne peuvent être traitées en silo mais doivent irriguer tout le projet.

Dans un contexte de données de santé, ces deux volets prennent une importance particulière : la supervision garantit la disponibilité attendue d'un service de soin, tandis que la sécurité protège des informations sensibles et contribue au respect du cadre réglementaire. Mon travail visait donc autant la performance opérationnelle que la conformité.

2. Ma contribution détaillée

2.1 Collecte des métriques (Prometheus)

J'ai déployé Prometheus, qui collecte périodiquement les métriques exposées par les différents composants. L'application publie ses propres indicateurs — nombre de requêtes, latence, code de réponse — que j'ai fait découvrir automatiquement par Prometheus grâce à un objet de configuration dédié. Ainsi, dès qu'une nouvelle instance de l'application démarre, elle est automatiquement prise en compte par la supervision, sans configuration manuelle.

J'ai veillé à limiter la cardinalité des métriques, c'est-à-dire à éviter une explosion du nombre de séries temporelles, en agrégeant les mesures par route plutôt que par requête individuelle. Ce souci rejoint directement la démarche FinOps de l'équipe, car une supervision mal maîtrisée peut consommer énormément de mémoire et de stockage, au point de coûter plus cher que l'application elle-même.

2.2 Visualisation (Grafana) et tableaux de bord

J'ai déployé Grafana et conçu un tableau de bord dédié à l'application, présentant les indicateurs clés : le débit de requêtes par route, la latence au 95e centile, le taux d'erreurs serveur et la disponibilité du backend. Ces quatre indicateurs correspondent aux « signaux dorés » de la supervision d'un service, reconnus comme les plus révélateurs de son état de santé.

J'ai automatisé l'import de ce tableau de bord afin qu'il soit présent dès l'installation, sans aucune manipulation. J'ai également exposé Grafana de façon accessible via l'Ingress existant, en réutilisant l'adresse publique du cluster grâce à un nom d'hôte dynamique — un choix qui évite de provisionner une seconde adresse IP publique et participe donc concrètement à la maîtrise des coûts, illustrant que sécurité, observabilité et FinOps se pensent ensemble.

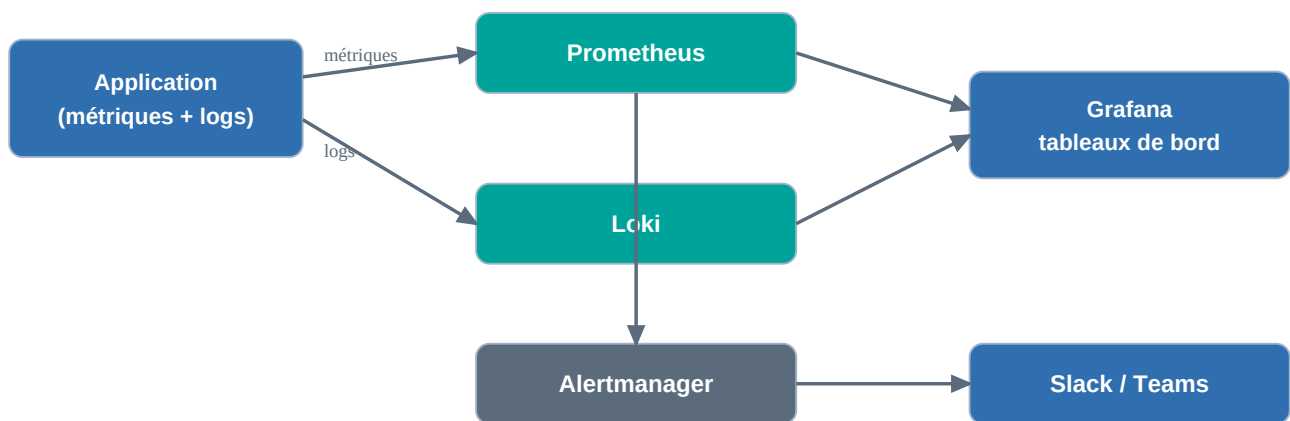
2.3 Centralisation des journaux (Loki)

Pour les journaux, j'ai déployé Loki accompagné de son agent de collecte, qui récupère les traces de tous les composants et les rend consultables et corrélables depuis Grafana, au même endroit que les métriques. L'application produit des journaux structurés, ce qui facilite leur exploitation et leur filtrage.

Le choix de Loki plutôt qu'une pile Elasticsearch a été motivé, en accord avec l'équipe, par son empreinte réduite : là où Elasticsearch indexe l'intégralité du contenu et réclame plusieurs gigaoctets de mémoire, Loki n'indexe que les étiquettes et se contente de ressources modestes, pour un usage — centralisation, recherche, visualisation — équivalent dans notre contexte. Ce choix a été déterminant pour tenir dans le budget.

2.4 Alerting (Alertmanager)

J'ai défini des règles d'alerte qui se déclenchent lorsque le service devient indisponible, lorsque le taux d'erreurs dépasse un seuil ou lorsque la latence se dégrade durablement. Ces alertes sont routées par Alertmanager vers une messagerie d'équipe (Slack ou Teams) au moyen d'un point d'entrée dédié. L'objectif est de prévenir l'équipe d'un incident avant même que les utilisateurs ne s'en aperçoivent, ce qui transforme une supervision passive en un dispositif réellement proactif.

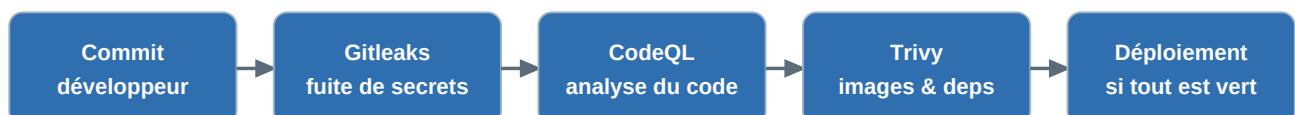


Chaîne d'observabilité : l'application émet métriques et journaux, collectés par Prometheus et Loki, visualisés dans Grafana ; les alertes sont routées par Alertmanager vers la messagerie de l'équipe.

2.5 Sécurité de la chaîne (DevSecOps)

J'ai intégré dans la chaîne d'intégration trois analyses complémentaires. Trivy examine les dépendances, les fichiers d'infrastructure et les images à la recherche de vulnérabilités connues, et bloque le déploiement en cas de faille critique. CodeQL réalise une analyse statique du code source pour détecter des schémas potentiellement dangereux. Gitleaks recherche toute fuite de secret dans l'historique du dépôt. Les résultats sont publiés dans l'onglet de sécurité du dépôt, au format standard SARIF, ce qui permet un suivi structuré.

Le fait d'exécuter ces analyses à chaque livraison, et non ponctuellement, matérialise le principe du « décalage vers la gauche » (*shift-left*) : les problèmes de sécurité sont détectés au plus tôt, quand ils sont les moins coûteux à corriger, plutôt qu'une fois le système en production. Les trois contrôles s'insèrent comme autant de barrières successives sur le trajet du code vers la production :



Portes de sécurité successives : un secret exposé, une faille de code ou une vulnérabilité critique d'image bloque la chaîne avant toute mise en production.

2.6 Durcissement de l'exécution

Au-delà des analyses, j'ai durci l'exécution des conteneurs : ils s'exécutent sans privilèges et sans possibilité d'élévation, les capacités système superflues sont retirées, et l'accès à l'API du cluster est régi par des rôles restreints selon le principe du moindre privilège. Les communications avec la base sont chiffrées en TLS, et les secrets ne sont jamais écrits dans le dépôt : ils transitent chiffrés et sont injectés dans les conteneurs au dernier moment.

J'ai également préparé des politiques de cloisonnement réseau limitant les communications entre composants au strict nécessaire, de sorte qu'une éventuelle compromission d'un pod ne puisse pas se propager librement dans le cluster. Ces politiques constituent une défense en profondeur, complémentaire des autres mesures.

3. Choix technologiques : pourquoi ces technologies plutôt que d'autres

Mon périmètre reposait sur deux familles d'outils : la pile d'observabilité et les analyseurs de sécurité. Chacune a été choisie après comparaison explicite avec ses alternatives, en pesant systématiquement le service rendu contre l'empreinte en ressources — un critère décisif compte tenu du budget.

3.1 Journalisation : Loki plutôt qu'Elasticsearch (pile ELK)

Critère	Loki (retenu)	Elasticsearch / ELK
Indexation	Étiquettes seulement	Contenu intégral des logs
Mémoire requise	Modeste (quelques centaines de Mo)	Plusieurs Go de RAM
Stockage	Compressé, peu volumineux	Index volumineux
Intégration Grafana	Native, source de données directe	Via Kibana (interface séparée)
Adéquation au budget	Excellente	Nécessiterait des nœuds plus gros

J'ai retenu **Loki** plutôt qu'une pile Elasticsearch parce qu'il n'indexe que les étiquettes des journaux, là où Elasticsearch indexe l'intégralité du contenu et réclame plusieurs gigaoctets de mémoire. Pour un usage identique dans notre contexte — centralisation, recherche, visualisation dans Grafana — Loki offre une empreinte très inférieure, ce qui a été déterminant pour tenir sur des nœuds modestes. Elasticsearch aurait imposé des machines plus puissantes et donc bien plus coûteuses, sans bénéfice tangible à notre échelle.

3.2 Métriques : Prometheus plutôt qu'Azure Monitor

Critère	Prometheus (retenu)	Azure Monitor
Modèle	Open source, standard cloud-native	Service managé propriétaire
Coût	Gratuit (auto-hébergé)	Facturé à l'ingestion / rétention
Portabilité	Indépendant du cloud	Lié à Azure
Découverte des cibles	Automatique (ServiceMonitor)	Agents / configuration Azure
Intégration Grafana	Native	Possible mais moins directe

Prometheus s'est imposé comme le standard open source de la métrologie cloud-native, avec une découverte automatique des cibles et une intégration native à Grafana. Azure Monitor, bien qu'intégré à la plateforme, facture l'ingestion et la rétention des données et enferme la supervision dans l'écosystème Azure ; Prometheus est gratuit, portable et parfaitement adapté à Kubernetes. Le triptyque Prometheus / Grafana / Loki forme ainsi un ensemble cohérent où l'exploitant corrèle en un seul endroit courbes et journaux.

3.3 Analyse de sécurité : Trivy, CodeQL et Gitleaks plutôt que des alternatives

Besoin	Outil retenu	Alternative écartée	Raison du choix
Images & dépendances	Trivy	Clair, Snyk	Gratuit, rapide, couvre IaC + images
Analyse du code	CodeQL	SonarQube	Intégré à GitHub, sans serveur
Fuite de secrets	Gitleaks	TruffleHog	Léger, simple à intégrer en CI

Pour la sécurité, j'ai retenu des outils gratuits, largement adoptés et complémentaires : **Trivy** pour les dépendances, les fichiers d'infrastructure et les images, **CodeQL** pour le code et **Gitleaks** pour les secrets. Snyk et SonarQube, plus riches, imposent respectivement un abonnement et un serveur à héberger ; Clair est plus limité que Trivy sur l'infrastructure as code. Cette combinaison couvre l'essentiel de la surface d'exposition sans coût de licence ni serveur supplémentaire, ce qui était impératif dans notre cadre. Intégrer ces contrôles directement dans la chaîne, plutôt que de les laisser à la discrétion de chacun, garantit enfin que la sécurité devient une propriété systématiquement vérifiée.

4. Difficultés rencontrées et solutions apportées

La principale difficulté a été de faire tenir une pile de supervision complète sur un cluster volontairement réduit pour des raisons budgétaires. J'ai dû dimensionner soigneusement les ressources allouées à chaque composant — Prometheus, Grafana, Loki, Alertmanager — et accepter des durées de rétention courtes, afin que la supervision n'entre pas en concurrence avec l'application pour les ressources limitées du cluster.

Une autre difficulté a concerné la découverte automatique des métriques de l'application. Il a fallu s'assurer que Prometheus sélectionne bien l'objet de configuration produit par le déploiement, ce qui supposait un bon ordonnancement de l'installation : les définitions de ressources nécessaires devaient précéder l'application. Ce point, situé à la frontière avec le périmètre d'Elyess, a été résolu en réorganisant l'ordre des étapes de la chaîne.

Enfin, exposer Grafana sans provisionner de ressource réseau supplémentaire a demandé de réutiliser l'Ingress existant avec un nom d'hôte dynamique : une solution élégante, mais qu'il a fallu mettre au point après avoir constaté qu'une exposition directe consommait une adresse IP publique supplémentaire, contraire à notre objectif de sobriété.

5. Perspectives d'évolution

Je souhaiterais ajouter le **traçage distribué**, au moyen d'OpenTelemetry et d'un outil comme Tempo ou Jaeger, afin de suivre une requête à travers les composants et de diagnostiquer finement les lenteurs. Ce troisième pilier de l'observabilité, complémentaire des métriques et des journaux, deviendrait particulièrement utile si l'application se complexifiait ou se découpait en services.

Je centraliserais également la gestion des secrets dans un coffre dédié offrant une rotation automatique des identifiants, ce qui réduirait la fenêtre d'exposition en cas de fuite et allégerait leur gestion manuelle.

Je renforcerais par ailleurs la posture de sécurité du cluster par des politiques avancées : standards de sécurité des pods, contrôleur d'admission validant automatiquement chaque manifeste, et signature des images pour garantir leur provenance et empêcher le déploiement d'artefacts non vérifiés.

Enfin, je définirais des objectifs de niveau de service (SLO) assortis d'un suivi de leur consommation, afin de piloter la fiabilité de manière quantifiée et d'objectiver les priorités d'amélioration.

6. Analyse critique des limites

Le routage des alertes vers une messagerie nécessite un point d'entrée externe qui n'a pas été connecté en production dans le cadre du projet : le mécanisme est en place et fonctionnel, mais sa validation de bout en bout, jusqu'à la réception effective d'une notification, reste à finaliser.

De même, les politiques de cloisonnement réseau que j'ai préparées supposent un module réseau capable de les appliquer, que nous n'avons pas activé sur le cluster d'étude afin de rester légers et économes : elles sont donc fournies et prêtes, mais non strictement appliquées dans la configuration actuelle, ce qui constitue un écart assumé entre la conception et l'exécution.

Enfin, une véritable conformité à l'hébergement de données de santé imposerait une région et des services certifiés que nous n'avons pas retenus dans ce cadre, et la rétention volontairement courte des métriques et des journaux limite l'analyse rétrospective des incidents les plus anciens.

7. Annexe — documentation utilisateur (mon périmètre)

7.1 Consulter la supervision

L'utilisateur accède à Grafana via son adresse dédiée, avec les identifiants fournis. Le tableau de bord « Vue d'ensemble » est disponible immédiatement et présente l'état de santé de l'application. Pour observer l'évolution des courbes, il suffit de générer un peu de trafic sur l'API : le débit, la latence et le taux d'erreurs se mettent alors à jour en temps quasi réel.

7.2 Explorer les journaux

Depuis Grafana, la vue d'exploration permet d'interroger la source de données Loki et de filtrer les journaux par composant ou par période, ce qui est précieux pour diagnostiquer un comportement anormal et remonter à sa cause sans se connecter directement aux conteneurs.

7.3 Consulter les rapports de sécurité

Les résultats des analyses de sécurité sont consultables dans l'onglet « Security » du dépôt, où chaque vulnérabilité détectée est listée avec sa gravité et sa localisation. Cette présentation permet de suivre l'évolution de la posture de sécurité dans le temps et de prioriser les corrections selon leur criticité.

7.4 Réagir à une alerte

Lorsqu'une règle se déclenche, une notification est émise vers la messagerie de l'équipe. La marche à suivre consiste alors à ouvrir le tableau de bord correspondant pour qualifier l'incident, puis à consulter les journaux associés afin d'en identifier l'origine avant d'engager la correction.

8. Analyse personnelle

Défis rencontrés

Mon principal défi a été de concilier une observabilité riche et une sécurité exigeante avec la contrainte de sobriété du projet. Il m'a fallu arbitrer en permanence entre l'exhaustivité souhaitable et ce que le cluster pouvait raisonnablement supporter, sans jamais sacrifier l'essentiel.

J'ai aussi dû dialoguer étroitement avec les autres membres, car la sécurité et la supervision touchent à tout : instrumenter l'application, ordonner la chaîne, sécuriser les objets Kubernetes supposaient un travail concerté plutôt qu'isolé.

Forces

Je retiens mon sens du détail sur les questions de sécurité et de conformité, ainsi que ma capacité à rendre un système transparent et compréhensible pour l'ensemble de l'équipe.

J'ai également su intégrer la dimension économique dans mes choix, en préférant systématiquement les solutions les plus sobres à service équivalent, ce qui a contribué à l'équilibre budgétaire global.

Faiblesses

Je dois concrétiser jusqu'au bout certaines mises en œuvre que j'ai préparées mais pas totalement activées, comme le routage effectif des alertes ou l'application stricte des politiques réseau.

J'ai parfois privilégié la mise en place d'un dispositif sur sa validation finale, ce qui laisse quelques maillons à éprouver de bout en bout avant de les considérer comme pleinement opérationnels.

Compétences développées

J'ai approfondi ma maîtrise de Prometheus, Grafana et Loki, ainsi que des outils DevSecOps que sont Trivy, CodeQL et Gitleaks, et des mécanismes de durcissement de Kubernetes.

Surtout, j'ai intégré la sécurité et l'observabilité comme des réflexes de conception, présents dès les premières décisions d'architecture, et non comme des ajouts de dernière minute.

Axes d'amélioration personnels

Je souhaite mettre en œuvre le traçage distribué et des objectifs de niveau de service, afin de compléter les deux piliers déjà couverts et de piloter la fiabilité de façon quantifiée.

Je compte également industrialiser la gestion des secrets et la signature des images, et automatiser la validation de bout en bout de la chaîne d'alerte pour fournir une observabilité complète et éprouvée.